

COMPTE RENDU

Algorithmique Avancée - Projet I - Compression de Huffman
3e année Cybersécurité - École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
Filière : Cybersécurité
Année : 2025/2026
Enseignant : M. BAKHOUYA
Date : 29 novembre 2025

Remerciements

Ce projet a été réalisé dans le cadre du cours d'Algorithmique Avancée et Structures de Données dispensé par le **Prof. Mohamed BAKHOUYA** à l'UIR durant l'année universitaire 2025-2026.

Je tiens à remercier le Prof. BAKHOUYA pour la qualité de son enseignement, notamment le Chapitre 2 (AG3.pdf) sur les arbres et les files de priorité, qui a constitué la base théorique essentielle de ce projet. Les travaux pratiques (TP4 sur les ABR, TP5 sur les Tas, TP6 sur le Tri par Tas, et TP7 sur les Files de Priorité) ont permis de consolider les concepts nécessaires à l'implémentation de l'algorithme de Huffman.

Je souhaite également remercier les enseignants des formations antérieures dont les cours ont constitué les fondations de ce travail :

- Les professeurs de l'**Université Grenoble Alpes** (INF301, INF304, INF401) pour la rigueur en programmation et algorithmique
- Les professeurs de **Polytech Grenoble** (PROG C, API, CPS, ALG, SystEmb, ALM) pour la maîtrise approfondie du langage C, des structures de données et de l'architecture logicielle

Leur enseignement m'a permis d'aborder ce projet avec les compétences techniques nécessaires.

Mohammed Taha HATHOUTI
Novembre 2025

Table des matières

1	Introduction	4
1.1	Contexte	4
1.2	Rappel de la consigne	4
1.3	Objectifs du projet	4
2	Choix de conception et justifications	4
2.1	Pourquoi l'algorithme de Huffman ?	4
2.2	Choix de la structure de données : Tas-Min	5
2.3	Architecture modulaire	6
2.4	Choix d'implémentation : Stockage en caractères '0'/'1'	6
3	Implémentation	6
3.1	Vue d'ensemble	6
3.2	Structures de données clés	7
3.2.1	NoeudHuffman	7
3.2.2	TasMin	7
3.3	Algorithmes principaux	7
3.3.1	Construction de l'arbre (complexité $O(n \log n)$)	7
3.3.2	Génération des codes (complexité $O(n)$)	7
3.3.3	Compression (complexité $O(n)$)	7
3.3.4	Décompression (complexité $O(n)$)	8
4	Résultats expérimentaux et analyse	8
4.1	Résultats de compression	8
4.2	Analyse des résultats	8
4.2.1	Cas optimal : fichier binaire (81.90%)	8
4.2.2	Cas très favorable : alphabet répété (75%)	8
4.2.3	Cas favorable : texte répétitif (69.26%)	9
4.2.4	Cas défavorable : distribution uniforme (39.81%)	9
4.3	Cas limites et pathologiques (<i>on négligera $\backslash n$</i>)	10
4.3.1	Fichier vide	10

4.3.2	Fichier mono-caractère	10
4.3.3	Fichier avec 2 caractères	10
4.4	Vérification d'intégrité	10
5	Discussion critique et limites	10
5.1	Limites de l'implémentation actuelle	10
5.1.1	Stockage inefficace (caractères vs bits)	10
5.1.2	Table des codes séparée	11
5.2	Limites théoriques de Huffman	11
5.2.1	Symboles indépendants	11
5.3	Améliorations possibles	11
5.3.1	Court terme (implémentation)	11
5.3.2	Moyen terme (algorithmes)	11
5.3.3	Long terme (recherche avancée)	11
6	Conclusion	12
6.1	Objectifs atteints	12
6.2	Réflexion finale	12

1 Introduction

1.1 Contexte

La compression de données est un domaine fondamental de l'informatique moderne, permettant de réduire la taille des fichiers pour optimiser leur stockage et transmission. Avec l'explosion des données numériques, les algorithmes de compression sont devenus indispensables dans de nombreux domaines : archivage, télécommunications, multimédia, etc.

Parmi les méthodes de compression sans perte, l'algorithme de **Huffman** (1952) occupe une place particulière en raison de sa simplicité, de son efficacité et de son optimalité pour l'encodage de symboles indépendants. Cet algorithme exploite les **fréquences d'apparition** des caractères pour construire un code binaire à longueur variable, attribuant des codes courts aux caractères fréquents et des codes longs aux caractères rares.

1.2 Rappel de la consigne

Le projet consiste à développer un programme fonctionnel de compression/décompression basé sur l'algorithme de Huffman. La consigne impose l'implémentation complète de :

- Une structure d'arbre binaire avec feuilles (caractères) et nœuds internes
- Un tas minimum (file de priorité) pour la construction efficace de l'arbre
- Les opérations fondamentales sur le tas : insertion, extraction, entassement
- La lecture et l'analyse des fréquences depuis un fichier source
- L'encodage et le décodage complets avec sauvegarde des résultats
- Des fonctions de test et de validation

1.3 Objectifs du projet

Les objectifs principaux sont :

1. **Comprendre et implémenter** l'algorithme de Huffman dans sa totalité
2. **Maîtriser** les structures de données avancées (arbres binaires, tas-min)
3. **Analyser expérimentalement** les performances de compression
4. **Produire** un code robuste et bien documenté
5. **Valider** l'implémentation sur des jeux de tests variés

2 Choix de conception et justifications

2.1 Pourquoi l'algorithme de Huffman ?

Le choix de Huffman pour ce projet pédagogique se justifie par plusieurs raisons :

1. Simplicité conceptuelle

L'algorithme repose sur une idée intuitive : attribuer des codes courts aux symboles fréquents. Cette approche "gloutonne" (greedy) est facile à comprendre et à implémenter, tout en introduisant des concepts algorithmiques importants.

2. Optimalité garantie

Huffman est **optimal** pour l'encodage de symboles indépendants : aucun autre code préfixe ne peut produire une longueur moyenne de code inférieure. Cette propriété mathématique fait de Huffman une référence théorique.

3. Applications réelles

Huffman est utilisé dans de nombreux formats de compression (JPEG, MP3, ZIP), ce qui confère au projet une pertinence pratique.

4. Pédagogie

Le projet permet d'explorer plusieurs concepts fondamentaux :

- Arbres binaires et parcours récursifs
- Files de priorité (tas-min)
- Complexité algorithmique

2.2 Choix de la structure de données : Tas-Min

Pour construire l'arbre de Huffman, il faut extraire répétitivement les deux nœuds de fréquence minimale. Deux approches sont possibles :

Structure	Extraction Min	Insertion
Liste triée	$O(1)$	$O(n)$
Tas-min	$O(\log n)$	$O(\log n)$

TABLE 1 – Comparaison liste triée vs tas-min

Complexité totale de l'algorithme :

- Avec liste triée : $O(n) + n \times O(n) = O(n^2)$
 - Avec tas-min : $O(n) + n \times O(\log n) = O(n \log n)$
- où n est le nombre de caractères distincts.

Le **tas-min** est donc choisi pour son efficacité asymptotique supérieure, particulièrement important pour des alphabets de grande taille.

Note importante sur le tas-min : Le tas-min est une structure **temporaire** utilisée uniquement pendant la phase de construction. Dans cette structure, les nœuds de petite fréquence remontent vers la racine du tas (propriété : $\text{freq}[\text{parent}(i)] \leq \text{freq}[i]$).

Cependant, dans l'**arbre de Huffman final**, la situation est inversée : les caractères les plus fréquents se retrouvent **proches de la racine** (codes courts), tandis que les caractères rares sont **profonds** dans l'arbre (codes longs). Cette inversion découle du principe même de l'algorithme : en fusionnant d'abord les nœuds de petite fréquence, ceux-ci se retrouvent naturellement éloignés de la racine finale.

2.3 Architecture modulaire

Le projet adopte une architecture en 7 modules indépendants :

Module	Justification
<code>types.h</code>	Centralisation des structures de données
<code>tas_min</code>	Abstraction de la file de priorité
<code>frequencies</code>	Séparation calcul statistique / construction arbre
<code>huffman</code>	Cœur de l'algorithme (construction + codes)
<code>compression</code>	Logique d'encodage isolée
<code>décodage</code>	Logique de décodage isolée
<code>modes</code>	Interface utilisateur séparée de la logique

TABLE 2 – Modules et justifications

Avantages de cette architecture :

- **Testabilité** : chaque module peut être testé indépendamment
- **Maintenabilité** : modifications localisées sans impact global
- **Réutilisabilité** : les modules (`tas-min`, etc.) peuvent servir ailleurs
- **Lisibilité** : séparation claire des responsabilités

2.4 Choix d'implémentation : Stockage en caractères '0'/'1'

Le projet stocke les bits compressés sous forme de caractères '0' et '1' plutôt que de vrais bits. Ce choix a été fait pour des raisons **pédagogiques** :

Avantages :

- **Débogage facile** : visualisation directe du résultat de compression
- **Portabilité** : pas de manipulation bit-à-bit complexe
- **Simplicité** : focus sur l'algorithme plutôt que sur les détails techniques
- **Validation** : vérification manuelle des codes générés (avec un `diff fichier_test.txt fichier_decode.txt`)

Inconvénient :

- Facteur 8 de perte d'efficacité (1 caractère = 8 bits au lieu de 1 bit)

Cette limitation pourrait être acceptable dans un contexte pédagogique où la compréhension et la manipulation prime sur l'optimisation réelle maximale.

3 Implémentation

3.1 Vue d'ensemble

Le projet comprend :

- **18 fichiers sources** (`.c/.h`) organisés en 7 modules
- **1 programme principal** avec 2 modes (interactif et fichier)
- **4 programmes de test** unitaires (`tas`, `huffman`, `compression`, `décodage`)
- **19 fichiers de test** couvrant les cas normaux et limites
- **1 Makefile** pour la compilation automatisée

3.2 Structures de données clés

3.2.1 NoeudHuffman

Représente un nœud de l'arbre binaire :

```
typedef struct NoeudHuffman {
    char caractere;           /* Symbole (si feuille) */
    int frequence;            /* Nombre d'occurrences */
    struct NoeudHuffman* gauche; /* Fils gauche */
    struct NoeudHuffman* droit;  /* Fils droit */
    int est_feuille;          /* 1 si feuille, 0 sinon */
} NoeudHuffman;
```

3.2.2 TasMin

Implémente la file de priorité :

```
typedef struct {
    NoeudHuffman** noeuds; /* Tableau de pointeurs */
    int taille;             /* Nombre d'elements */
    int capacite;           /* Capacite maximale */
} TasMin;
```

3.3 Algorithmes principaux

3.3.1 Construction de l'arbre (complexité $O(n \log n)$)

L'algorithme suit ces étapes :

1. Créer un tas-min avec tous les caractères comme feuilles
2. Tant que le tas contient plus d'un nœud :
 - Extraire les deux nœuds de fréquence minimale
 - Créer un nœud parent avec $\text{freq} = \text{freq}_{\text{gauche}} + \text{freq}_{\text{droit}}$
 - Réinsérer le parent dans le tas
3. Le dernier nœud restant est la racine

3.3.2 Génération des codes (complexité $O(n)$)

Parcours en profondeur (DFS) de l'arbre :

- Descendre à gauche → ajouter '0' au code
- Descendre à droite → ajouter '1' au code
- Atteindre une feuille → enregistrer le code

3.3.3 Compression (complexité $O(n)$)

Pour chaque caractère du fichier source, remplacer par son code binaire selon la table.

3.3.4 Décompression (complexité $O(n)$)

Parcourir l'arbre selon les bits lus : '0' → gauche, '1' → droite. Atteindre une feuille → écrire le caractère et revenir à la racine.

4 Résultats expérimentaux et analyse

4.1 Résultats de compression

Fichier	Orig. (bits)	Comp. (bits)	Gain	Taux
simple.txt	136	59	77	56.62%
repetitif.txt	296	91	205	69.26%
long.txt	2320	1201	1119	48.23%
uniforme.txt	216	130	86	39.81%
court.txt	152	41	111	73.03%
alphabet.txt	128	32	96	75.00%
binaire.txt	232	42	190	81.90%

TABLE 3 – Résultats de compression sur différents types de fichiers

4.2 Analyse des résultats

4.2.1 Cas optimal : fichier binaire (81.90%)

Le fichier `binaire.txt` contient uniquement des caractères '0', '1' et '\n' :

```
0000111100001111000011110000\n
```

Distribution des fréquences :

- '0' : 16 occurrences → code "1" (1 bit)
- '1' : 12 occurrences → code "01" (2 bits)
- '\n' : 1 occurrence → code "00" (2 bits)

Avec seulement **3 symboles**, Huffman génère des codes très courts (1-2 bits) contre 8 bits en ASCII. La distribution inégale (55% de '0') permet d'optimiser davantage : le caractère dominant reçoit le code le plus court (1 bit). Le gain est exceptionnel : de 232 bits (ASCII) à 42 bits (Huffman), soit une réduction de $5.5\times$.

4.2.2 Cas très favorable : alphabet répété (75%)

Le fichier `alphabet.txt` contient :

```
ABCABCABCABCABC\n
```

(motif ABC répété 5 fois)

Distribution des fréquences :

- 'A' : 5 occurrences → code "11" (2 bits)
- 'B' : 5 occurrences → code "10" (2 bits)
- 'C' : 5 occurrences → code "01" (2 bits)
- '\n' : 1 occurrence → code "00" (2 bits)

Malgré une distribution **quasi-uniforme** entre les 3 caractères principaux, le taux de compression atteint 75%. Cela s'explique par le fait que 4 symboles nécessitent exactement $\lceil \log_2(4) \rceil = 2$ bits, contre 8 bits en ASCII. Huffman génère des codes de longueur constante (2 bits chacun), ce qui donne : $16 \times 2 = 32$ bits au lieu de $16 \times 8 = 128$ bits, soit une réduction de 4×.

4.2.3 Cas favorable : texte répétitif (69.26%)

Le fichier `repetitif.txt` contient :

```
AAAAAAAAAAAAABBBBBBBBCCCCCDDDDDEEEEE\n
```

Distribution des fréquences :

- 'A' : 12 occurrences (32%) → code "11" (2 bits)
- 'B' : 8 occurrences (22%) → code "01" (2 bits)
- 'C' : 6 occurrences (16%) → code "00" (2 bits)
- 'E' : 5 occurrences (14%) → code "100" (3 bits)
- 'D' : 5 occurrences (14%) → code "1011" (4 bits)
- '\n' : 1 occurrence (2%) → code "1010" (4 bits)

La **forte inégalité** des fréquences permet à Huffman d'exploiter efficacement la redondance. Les 3 caractères les plus fréquents (A, B, C) représentent 70% du fichier et reçoivent des codes courts (2 bits), tandis que les caractères rares (D, \n) obtiennent des codes longs (4 bits). Cette stratégie minimise la longueur totale : 91 bits au lieu de 296 bits, d'où le taux de 69.26%.

4.2.4 Cas défavorable : distribution uniforme (39.81%)

Le fichier `uniforme.txt` contient l'alphabet complet :

```
abcdefghijklmnopqrstuvwxyz\n
```

Problème : Toutes les lettres apparaissent exactement 1 fois (+ 1 retour ligne). Avec une distribution **parfaitement uniforme**, Huffman ne peut exploiter aucune inégalité. Les codes générés ont des longueurs très proches : 5 symboles à 4 bits, 22 symboles à 5 bits.

Longueur moyenne : $(5 \times 4 + 22 \times 5) / 27 = 130 / 27 \approx 4.81$ bits/symbole

Comparaison avec ASCII :

- ASCII : 8 bits/caractère $\times 27 = 216$ bits
- Huffman : 130 bits
- Gain : 39.81%

Ce résultat illustre la **limite de Huffman** : sans redondance ni inégalité des fréquences, la compression reste modeste.

4.3 Cas limites et pathologiques (*on négligera $\backslash n$*)

4.3.1 Fichier vide

Le programme détecte correctement le cas et refuse la compression (aucune donnée à encoder).

4.3.2 Fichier mono-caractère

Pour un fichier contenant uniquement "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA" :

- Un seul nœud dans le tas
- Arbre réduit à une seule feuille
- Code arbitraire attribué (ex : "0")
- Compression réussie mais peu significative

4.3.3 Fichier avec 2 caractères

Pour "ABABABABABABABABABAB" :

- A et B ont la même fréquence
- Codes : A="0", B="1"
- Taux de compression : 87.5% (8 bits $\rightarrow$ 1 bit par symbole)

4.4 Vérification d'intégrité

Le programme effectue systématiquement :

1. Compression du fichier source
2. Décompression du fichier compressé
3. Comparaison caractère par caractère

Résultat : 100% de réussite sur les 19 fichiers de test, confirmant la **compression sans perte**.

5 Discussion critique et limites

5.1 Limites de l'implémentation actuelle

5.1.1 Stockage inefficace (caractères vs bits)

L'implémentation stocke les bits sous forme de caractères '0' et '1', ce qui consomme **8 fois plus d'espace** que nécessaire.

Impact : Les taux de compression affichés sont théoriques. En pratique, avec l'implémentation actuelle, le fichier compressé est souvent **plus gros** que l'original.

Exemple :

- Fichier original : 29 caractères = 29 octets
- Fichier "compressé" (stockage '0'/'1') : 42 caractères = 42 octets
- Paradoxe : le fichier compressé est $\approx 1,45\times$ plus gros !

Solution : Implémenter un vrai encodage bit-à-bit avec manipulation des octets.

5.1.2 Table des codes séparée

La table de décodage est stockée dans un fichier séparé (`*_codes.txt`), ce qui pose deux problèmes :

1. **Gestion** : il faut transmettre deux fichiers (données + table)
2. **Overhead** : la table peut être volumineuse (plusieurs Ko)

Solution : Intégrer la table dans un header au début du fichier compressé, comme le font les formats réels (ZIP, GZIP).

5.2 Limites théoriques de Huffman

5.2.1 Symboles indépendants

Huffman traite chaque caractère indépendamment, ignorant les corrélations entre symboles.

Exemple : En français, 'q' est presque toujours suivi de 'u'. Huffman n'exploite pas cette information.

5.3 Améliorations possibles

5.3.1 Court terme (implémentation)

- Encodage bit-à-bit réel (gain $\times 8$) *en contradiction avec le côté pédagogique et insctructif de la matière*
- Support des fichiers binaires
- Compression de plusieurs fichiers

5.3.2 Moyen terme (algorithmes)

- **Huffman adaptatif** : mise à jour dynamique de l'arbre
- **Huffman canonique** : table plus compacte, décodage plus rapide

5.3.3 Long terme (recherche avancée)

- Parallélisation de la compression
- Optimisation cache-aware

6 Conclusion

Ce projet a permis l'implémentation complète et fonctionnelle de l'algorithme de compression de Huffman, depuis le calcul des fréquences jusqu'à la décompression avec vérification d'intégrité.

6.1 Objectifs atteints

Réalisations techniques :

- Architecture modulaire robuste (7 modules, 18 fichiers sources)
- Implémentation complète du tas-min avec toutes les opérations
- Construction de l'arbre de Huffman
- Génération des codes par parcours récursif
- Compression/décompression fonctionnelles
- Tests exhaustifs (19 fichiers, 100% de réussite)

Apports pédagogiques :

- Maîtrise des arbres binaires et parcours récursifs
- Compréhension approfondie des files de priorité (tas-min)
- Analyse de complexité ($O(n \log n)$ pour Huffman)

6.2 Réflexion finale

Au-delà de l'aspect technique, ce projet illustre l'importance de comprendre les **compromis** en informatique : simplicité vs performance, optimalité théorique vs efficacité pratique, généralité vs spécialisation.

Ce travail démontre également que la compression de données n'est pas une opération universelle : son efficacité dépend fondamentalement de la nature des données traitées. Huffman excelle sur des textes avec forte redondance (81.90% pour `binaire.txt`) mais reste modeste sur des distributions uniformes (39.81% pour `uniforme.txt`).

L'approche pédagogique adoptée — privilégier la compréhension à l'efficacité brute — s'est révélée particulièrement fructueuse. Le stockage en caractères '0'/'1', bien que inefficace en production, a facilité le débogage et la validation, permettant une meilleure assimilation des concepts fondamentaux.

Enfin, ce projet constitue une excellente introduction aux algorithmes gloutons et aux structures de données avancées (arbres, tas-min), tout en offrant une perspective concrète sur les défis réels de l'optimisation algorithmique.

Références

- **Cours** : Prof. Mohamed BAKHOUYA, *Algorithmique et Structures de Données Avancées*, UIR 2025-2026
- **Cours** : Prof. Gwenaël DELAVAL, Responsable de l'UE INF304, *Bases du développement logiciel : modularisation, tests - C*, L2 - S3, Université Grenoble Alpes (UGA) 2023-2024
- **Cours** : Prof. Florent BOUCHEZ TICHADOU, Responsable de l'UE INF301, *Algorithmique et Programmation Impérative*, L2 - S3, Université Grenoble Alpes (UGA) 2023-2024
- **Cours** : Prof. Denis BOUHINEAU, Responsable de l'UE INF401, *Architectures des ordinateurs*, L2 - S4, Université Grenoble Alpes (UGA) 2023-2024
- **Cours** : Prof. Michael PERIN, Responsables de filière des INFO3 et de l'UE PROG C, *Programmation C*, 3A - S5, Polytech Grenoble INP 2024-2025
- **Cours** : Prof. Vincent DENJEAN, Responsable de l'UE API, *Algorithmique et Programmation Impérative*, 3A - S5, Polytech Grenoble INP 2024-2025
- **Cours** : Prof. Eric GASCARD & Prof. Nicolas PALIX, Responsables de l'UE CPS, *C Programmation Systeme*, 3A - S5, Polytech Grenoble INP 2024-2025
- **Cours** : Prof. Pascal SICARD & Prof. Akram IDANI, Responsables de l'UE Sys-tEmb, *Systèmes Embarqués*, 3A - S5, Polytech Grenoble INP 2024-2025
- **Cours** : Prof. Pascal SICARD, Responsable de l'UE ALM, *Architectures Logicielles et Matérielles*, 3A - S6, Polytech Grenoble INP 2024-2025
- **Cours** : Prof. Eric GASCARD, Responsables de l'UE ALG, *Algorithmique Avancée*, 3A - S6, Polytech Grenoble INP 2024-2025
- **Guide** : M. Mohammed Taha HATHOUTI, *Les HEADERS en C*, UIR ESIN 2025-2026
- **The GNU C Reference Manual**, <https://www.gnu.org/software/gnu-c-manual/gnu-c-manual.html>
- **C Programming Wikibooks**, *The Free Textbook Project*, http://en.wikibooks.org/wiki/C_Programming
- **Cours** : Bernard CASSAGNE, du laboratoire **CLIPS** de Grenoble, maintenu par Matthieu MOY, *Introduction au langage C*, <http://matthieu-moy.fr/cours/poly-c/>, Université Joseph Fourier & cnrs
- **TPs** : TP4 (ABR), TP5 (Tas), TP6 (Tri), TP7 (Files de Priorité), UIR ESIN 2025-2026
- **Assistant IA** : Claude (Sonnet 4.5), Anthropic, <https://claude.ai>

Projet I - Algorithmique Avancée
Année universitaire 2025-2026